DKL / data-catalog-api

Code | Issues | Pull requests 12 | Actions | Projects | Wiki | Security

Edit

Datauser 6134 sonar codesmell fix part2 #1037

Open

mramadas121621 wants to merge 5 commits into develop from DATAUSER-6134-sonar-codesmell-fix-part2

Conversation 0 | Commits 5 | Checks 0 | Files changed 57

```
42  42          return field == null || StringUtils.isBlank(field);
43  43      }
44  44
45  -       public static Boolean checkIfFieldIsNull(List field) {
    45  +   public static <T> Boolean checkIfFieldIsNull(List<T> field) {
46  46          return field == null || field.isEmpty();
47  47      }
```

2 src/main/java/com/socgen/bigdata/catalog/commons/Tools.java

```
...  ...  @@ -1,12 +1,13 @@
1    1    package com.socgen.bigdata.catalog.enums;
2    -
     2  + import lombok.Getter;
     3  + @Getter
3    4    public enum DatasetRelationType {
4    5        PARENT("is the parent of"),
5    6        CHILD("is a child of"),
6    7        SUGGESTS("is connected with"),
7    8        SIMILAR("is similar to");
8    9
9    -        private String label;
     10 +     private final String label;
10   11
11   12       DatasetRelationType(String l) {
12   13           this.label = l;
30   31           throw new IllegalArgumentException(String.format("DatasetRelationType has
     not %s as code", value));
```

```
31  32      }
```

8 src/main/java/com/socgen/bigdata/catalog/enums/DatasetRelationType.java

```
35      -   }
36  34  }
```

```
...  ...  @@ -1,21 +1,20 @@
1    1    package com.socgen.bigdata.catalog.enums;
2    2
     3  + import lombok.Getter;
     4  +
     5  + @Getter
3    6    public enum Environments {
4    7        prd("1"),
5    8        uat("2"),
6    9        dev("3"),
7    10       default_weight("-1");
8    11
9        -    private String weight;
     12 +    private final String weight;
10   13
11   14       Environments(String weight) {
12   15           this.weight = weight;
13   16       }
14   17
15       -    public String getWeight() {
16       -        return weight;
17       -    }
```

9 src/main/java/com/socgen/bigdata/catalog/enums/Environments.java

```
21  20          if (e.name().equalsIgnoreCase(env)) {
```

```
...  ...  @@ -1,17 +1,20 @@
1    1    package com.socgen.bigdata.catalog.enums;
2    2
     3  + import lombok.Getter;
     4  +
3    5    /**
4    6     * Created by jbouahom122616 on 05/12/2017.
5    7     */
     8  + @Getter
6    9    public enum Exposition {
7    10       hidden("hidden", "Hidden"),
8    11       restricted("restricted", "Restricted"),
```

```
9   12      group("group", "Group"),
10  13      partner("partner", "Partner"),
11  14      ePublic("public", "Public");
12  15
13      -   private String displayName;
14      -   private String zeeneaName;
    16  +   private final String displayName;
    17  +   private final String zeeneaName;
15  18
16  19      Exposition(String displayName, String zeeneaName) {
17  20          this.displayName = displayName;
23  26          return displayName;
24  27      }
25  28
26      -   public String getZeeneaName() {
27      -       return zeeneaName;
28      -   }
29      -
30      -   public String getDisplayName() {
```

Changes from **all commits** File filter Conversations Jump to

```
35  30          for (Exposition exposition : Exposition.class.getEnumConstants()) {
36  31              if (exposition.toString().equals(myValue)) {
```

7 src/main/java/com/socgen/bigdata/catalog/enums/ExternalDataType.java

```
2   2
    3   + import lombok.Getter;
    4   +
    5   + @Getter
3   6   public enum ExternalDataType {
4   7       RAW("Raw"),
5   8       ESTIMATED_DATA("Estimated data"),
11  14          this.label = label;
12  15      }
13  16
14      -   public String getLabel() {
15      -       return label;
16      -   }
17      -
18  17      public static String getLabelFromEnumString(String text) {
19  18          for (ExternalDataType externalDataType : ExternalDataType.values()) {
20  19              if (externalDataType.name().equalsIgnoreCase(text)) {
```

9 src/main/java/com/socgen/bigdata/catalog/enums/FeedFrequency.java

```diff
@@ -1,8 +1,11 @@
package com.socgen.bigdata.catalog.enums;

+ import lombok.Getter;
+
/**
 * Created by jbouahom122616 on 05/12/2017.
 */
+ @Getter
public enum FeedFrequency {
    STREAMED("Streamed"),
    DAILY("Daily"),
    ANNUALLY("Annually"),
    OTHER("Other");

-    private String displayName;
+    private final String displayName;

    FeedFrequency(String displayName) {
        this.displayName = displayName;
        return displayName;
    }

-    public String getDisplayName() {
-        return displayName;
-    }
-
    public static FeedFrequency nameOfOrDefault(String name) {
        try {
            return name != null ? Enum.valueOf(FeedFrequency.class, name) : null;
```

4 src/main/java/com/socgen/bigdata/catalog/enums/Lifecycle.java

```diff
    trashed("Trashed", "Trashed"),
    deprecated("Deprecated", "Deprecated");

-    private String displayName;
-    private String zeeneaName;
+    private final String displayName;
+    private final String zeeneaName;

    Lifecycle(String displayName, String zeeneaName) {
        this.displayName = displayName;
```

2

src/main/java/com/socgen/bigdata/catalog/enums/PublicCloudEligibilityStatus.java

```diff
    CLOUD_AUTHORITY_AUTHORIZATION("Sensitive"),
    MISSING_INFORMATION("Missing information");

```

```
14        -    private String label;
     14   +    private final String label;
15   15
16   16        PublicCloudEligibilityStatus(String label) {
17   17            this.label = label;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/RoleAssignement.java

```
4    4         APPLICATION_DEVELOPMENT_MANAGER("Application development manager (SG)"),
5    5         BUSINESS_APPLICATION_MANAGER("Business application manager (SG)");
6    6
7         -    private String roleName;
     7   +    private final String roleName;
8    8
9    9         RoleAssignement(String roleName) {
10   10            this.roleName = roleName;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/SourceStatus.java

```
9    9         decommissioned("DECOMMISSIONED"),
10   10        closed("CLOSED");
11   11
12        -    private String displayName;
     12   +    private final String displayName;
13   13
14   14        SourceStatus(String displayName) {
15   15            this.displayName = displayName;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/StorageSpace.java

```
11   11        USAGE("Usage View");
12   12
13   13
14        -    private String label;
     14   +    private final String label;
15   15
16   16        StorageSpace(String label) {
17   17            this.label = label;
```

2

src/main/java/com/socgen/bigdata/catalog/enums/dataset/ClientDataCategories.java

```
9   9       INTERNAL_MANAGEMENT("Internal management (NBI, risks analysis)"),
10  10      NO_CLIENT_DATA_CATEGORIE("No client data");
11  11
12      -   private String label;
    12  +   private final String label;
13  13
14  14      ClientDataCategories(String label) {
15  15          this.label = label;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/dataset/DataGoldenNature.java

```
10  10      OTHER_DATALAKE_REPLICATION("Other data lake replication"),
11  11      NOT_GOLDEN_DATA("Not golden data");
12  12
13      -   private String label;
    13  +   private final String label;
14  14
15  15      DataGoldenNature(String label) {
16  16          this.label = label;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/dataset/DataOrigin.java

```
8   8       PUBLIC("Public (Open Data)"),
9   9       EXTERNAL("External (from third party)");
10  10
11      -   private String label;
    11  +   private final String label;
12  12
13  13      DataOrigin(String label) {
14  14          this.label = label;
```

2 src/main/java/com/socgen/bigdata/catalog/enums/dataset/GDPRClassification.java

```
11  11      DP2("DP2"),
12  12      DP3("DP3");
13  13
14      -   private String label;
    14  +   private final String label;
15  15
16  16      GDPRClassification(String label) {
17  17          this.label = label;
```

5 src/main/java/com/socgen/bigdata/catalog/enums/sort/EntitySortEnum.java

```
...  ...    @@ -1,10 +1,12 @@
1    1      package com.socgen.bigdata.catalog.enums.sort;
```

```
2   2
    3   + import lombok.Getter;
3   4     import org.springframework.data.domain.Sort;
4   5
5   6     /**
6   7      * Created by jbouahom122616 on 09/01/2018.
7   8      */
    9   + @Getter
8   10    public enum EntitySortEnum implements SortEnumInterface {
9   11        alphabetical(0),
10  12        recently(1),
30  32            }
31  33        }
32  34
33      -     public Sort getSort() {
34      -         return sort;
35      - }
36  35    }
```

5 src/main/java/com/socgen/bigdata/catalog/enums/sort/TagSortEnum.java

```
...  ...    @@ -1,10 +1,12 @@
1   1     package com.socgen.bigdata.catalog.enums.sort;
2   2
    3   + import lombok.Getter;
3   4     import org.springframework.data.domain.Sort;
4   5
5   6     /**
6   7      * Created by jbouahom122616 on 09/01/2018.
7   8      */
    9   + @Getter
8   10    public enum TagSortEnum implements SortEnumInterface {
9   11        alphabetical(0),
10  12        top(1);
24  26            }
25  27        }
26  28
27      -     public Sort getSort() {
28      -         return sort;
29      - }
30  29    }
```

2 src/main/java/com/socgen/bigdata/catalog/helpers/ExcelBuilderHelper.java

```
154 154                                 if (excelRowList != null && !excelRowList.isEmpty()) {
155 155                                     for (int j = rowHeaderIndex + 1; j <=
    excelRowList.size() + rowHeaderIndex; j++) {
156 156                                         Row dataRow = sheetExcel.createRow(j);
157     -                                       List cellValues = excelRowList.get(j -
    rowHeaderIndex - 1) != null
```

```
+                                   List<?> cellValues = excelRowList.get(j - rowHeaderIndex - 1) != null
                                        ? excelRowList.get(j - rowHeaderIndex - 1).getCellValues()
                                        : null;
```

10 src/main/java/com/socgen/bigdata/catalog/helpers/ExcelHelper.java

```
            workbook.close();
            return rowList;
        } catch (IOException e) {
-           throw new RuntimeException("Fail to parse Excel file: " + e.getMessage(), e);
+           throw new IllegalStateException("Fail to parse Excel file: " + e.getMessage(), e);
        }
    }

                            cellValue,
                            cellEx
                        );
-                       throw new RuntimeException(
+                       throw new IllegalStateException(
                            String.format(
                                "Error in sheet '%s' row %d column %d ('%s'): %s",
                                sheetName,

            return results;
        } catch (NoSuchMethodException | InstantiationException | IllegalAccessException | InvocationTargetException e) {
-           throw new RuntimeException("Fail to read data from Excel: " + e.getMessage(), e);
+           throw new IllegalStateException("Fail to read data from Excel: " + e.getMessage(), e);
        }
    }

                                cellValue,
                                cellEx
                            );
-                           throw new RuntimeException(
+                           throw new IllegalStateException(
                                String.format(
                                    "Error in business data sheet row %d column %d ('%s'): %s",
                                    rowNum + 1,
            }
            return mappings;
        } catch (RuntimeException e) {
-           throw new RuntimeException("Fail to read business data mappings from Excel: " + e.getMessage(), e);
```

```
+            throw new IllegalStateException("Fail to read business data mappings from Excel: " + e.getMessage(), e);
        }
    }

```

8 src/main/java/com/socgen/bigdata/catalog/helpers/HiveTableStructGenerator.java

```
@@ -1,12 +1,14 @@
package com.socgen.bigdata.catalog.helpers;

import java.util.ArrayList;
+import java.util.Collections;
import java.util.List;
import java.util.regex.Pattern;
import lombok.extern.slf4j.Slf4j;

@Slf4j
public class HiveTableStructGenerator {
+    private static final String TYPE_ARRAY="array";

    private HiveTableStructGenerator() {
        throw new IllegalStateException("Utility class");
                String keyType = keyValueSplit[0].trim();
                String valueType = keyValueSplit[1].trim();
                return String.format("map<%s,%s>", keyType, getComplexType(valueType));
-            case "array":
-                return String.format("array<%s>", getComplexType(removeTypeMarker(complexType, "array")));
+            case TYPE_ARRAY:
+                return String.format("array<%s>", getComplexType(removeTypeMarker(complexType, TYPE_ARRAY)));
            default:
                return null;
        }
            log.warn("Union type is not supported");
            return null;
        }
-        if ("array".equals(mainType)) {
+        if (TYPE_ARRAY.equals(mainType)) {
            return generateHiveColumnRecursively(nestedType, prefix);
        }
        if ("map".equals(mainType)) {
```

3 src/main/java/com/socgen/bigdata/catalog/helpers/JsonDtoCompare.java

```
                .flatMap(e -> flatten(new AbstractMap.SimpleEntry<>(entry.getKey() + "." + e.getKey(), e.getValue())));
        }

```

```
-         if (entry.getValue() instanceof List<?>) {
-             List<?> list = (List) entry.getValue();
+         if (entry.getValue() instanceof List<?> list) {
              return IntStream.range(0, list.size())
                  .mapToObj(i -> new AbstractMap.SimpleEntry<String, Object>(entry.getKey() + "." + i, list.get(i)))
                  .flatMap(JsonDtoCompare::flatten);
```

1 src/main/java/com/socgen/bigdata/catalog/helpers/PaginationHelper.java

```

  import com.socgen.apibank.http.util.PaginationUtil;
  import java.net.URISyntaxException;
+ import java.util.List;
  import lombok.extern.slf4j.Slf4j;
  import org.springframework.data.domain.Page;
  import org.springframework.http.HttpHeaders;
```

8

...main/java/com/socgen/bigdata/catalog/helpers/datasource/table/GenerateFromHiveHelper.java

```
13  13    import lombok.extern.slf4j.Slf4j;
14  14
15  15    @Slf4j
    16  +
16  17    public class GenerateFromHiveHelper {
    18  +     private static final String TYPE_ARRAY = "array";
17  19
18  20        private GenerateFromHiveHelper() {
19  21            /* This utility class should not be instantiated */
39  41                log.warn("Union type is not supported");
40  42                return Collections.emptyList();
41  43            }
42      -         if ("array".equals(mainType)) {
    44  +         if (TYPE_ARRAY.equals(mainType)) {
43  45                return generateDataSourceTableColumnRecursively(
44  46                    nestedType,
45  47                    parentDataSourceTableColumn,
195 197               String valueType = splitKeyValue[1].trim();
196 198
197 199               return String.format("map<%s,%s>", keyType,
    getComplexType(valueType));
198     -             case "array":
199     -                 return String.format("array<%s>",
    getComplexType(removeTypeMarker(complexType, "array")));
    200 +             case TYPE_ARRAY:
    201 +                 return String.format("array<%s>",
    getComplexType(removeTypeMarker(complexType, TYPE_ARRAY)));
200 202               default:
201 203                   return null;
202 204           }
```

2 src/main/java/com/socgen/bigdata/catalog/models/excel/ExcelColumn.java

```
21  21
22  22        public ExcelColumn(String label) {
23  23            this.label = label;
24      -         this.width = label != null ? (short) (label.toString().length() * 240) :
    3000;
    24  +         this.width = label != null ? (short) (label.length() * 240) : 3000;
25  25            if (this.width < 3000) {
26  26                this.width = 3000;
27  27            } else if (this.width > 22000) {
```

6 src/main/java/com/socgen/bigdata/catalog/models/excel/ExcelRow.java

```
10  10    @Data
11  11    public class ExcelRow {
```

```
12  12
13      -   private List cellValues;
    13  +   private List<?> cellValues;
14  14      private List<ExcelCellStyle> cellRowStyle;
15  15      private Short height;
16  16
17      -   public ExcelRow(List cellValues) {
    17  +   public ExcelRow(List<?> cellValues) {
18  18          this.cellValues = cellValues;
19  19      }
20  20
21      -   public ExcelRow(List cellValues, List<ExcelCellStyle> cellRowStyle) {
    21  +   public ExcelRow(List<?> cellValues, List<ExcelCellStyle> cellRowStyle) {
22  22          this(cellValues);
23  23          this.cellRowStyle = cellRowStyle;
24  24      }
```

12 src/main/java/com/socgen/bigdata/catalog/models/jpa/JpaAuditHistroy.java

```
51  51      public final boolean equals(Object o) {
52  52          if (this == o) return true;
53  53          if (o == null) return false;
54      -       Class<?> oEffectiveClass = o instanceof HibernateProxy
55      -           ? ((HibernateProxy) o).getHibernateLazyInitializer().getPersistentClass()
    54  +       Class<?> oEffectiveClass = (o instanceof HibernateProxy hp)
    55  +           ? (hp.getHibernateLazyInitializer().getPersistentClass())
56  56              : o.getClass();
57      -       Class<?> thisEffectiveClass = this instanceof HibernateProxy
58      -           ? ((HibernateProxy) this).getHibernateLazyInitializer().getPersistentClass()
    57  +       Class<?> thisEffectiveClass = (this instanceof HibernateProxy th)
    58  +           ? (th.getHibernateLazyInitializer().getPersistentClass())
59  59              : this.getClass();
60  60          if (thisEffectiveClass != oEffectiveClass) return false;
61  61          JpaAuditHistroy that = (JpaAuditHistroy) o;
64  64
65  65      @Override
66  66      public final int hashCode() {
67      -       return this instanceof HibernateProxy
68      -           ? ((HibernateProxy) this).getHibernateLazyInitializer().getPersistentClass().hashCode()
    67  +       return (this instanceof HibernateProxy hp)
    68  +           ? (hp.getHibernateLazyInitializer().getPersistentClass().hashCode())
69  69              : getClass().hashCode();
70  70      }
71  71  }
```

18

...main/java/com/socgen/bigdata/catalog/repositories/datasource/DataSourceSpecification.java

```
16  16
17  17  @Data
18  18  public class DataSourceSpecification implements Specification<DataSource> {
    19 +     private static final String HOSTING_PLATFORM = "hostingPlatform";
    20 +     public static final String APPLICATION = "application";
19  21
20  22      private DataSourceCriteria dataSourceCriteria;
21  23
91  93          }
92  94
93  95          return Optional.ofNullable(
94     -
        criteriaBuilder.in(root.get("hostingPlatform")).value(dataSourceCriteria.getHosting
        Platforms())
    96 +
        criteriaBuilder.in(root.get(HOSTING_PLATFORM)).value(dataSourceCriteria.getHostingP
        latforms())
95  97          );
96  98      }
97  99
101 103         }
102 104
103 105         return Optional.ofNullable(
104    -            criteriaBuilder.equal(root.get("hostingPlatform"),
        dataSourceCriteria.getHostingPlatform())
    106 +            criteriaBuilder.equal(root.get(HOSTING_PLATFORM),
        dataSourceCriteria.getHostingPlatform())
105 107         );
106 108     }
107 109
136 138
137 139         // For Snowflake: Including data sources where hostingPlatform is SNOWFLAKE
        and storageSpace is NULL
138 140         Predicate isSnowflakeWithNullStorage = criteriaBuilder.and(
139    -            criteriaBuilder.equal(root.get("hostingPlatform"),
        HostingPlatform.SNOWFLAKE),
    141 +            criteriaBuilder.equal(root.get(HOSTING_PLATFORM),
        HostingPlatform.SNOWFLAKE),
140 142             criteriaBuilder.isNull(root.get("storageSpace"))
141 143         );
142 144
161 163
162 164         return Optional.ofNullable(
163 165             criteriaBuilder.equal(
164    -                root.join("application", JoinType.LEFT).get("applicationId"),
    166 +                root.join(APPLICATION, JoinType.LEFT).get("applicationId"),
165 167                 dataSourceCriteria.getApplicationId()
166 168             )
167 169         );
174 176
175 177         return Optional.ofNullable(
176 178             criteriaBuilder.equal(
```

```
177      -                root.join("application", JoinType.LEFT).get("code"),
     179 +                root.join(APPLICATION, JoinType.LEFT).get("code"),
178  180                  dataSourceCriteria.getApplicationCode()
179  181              )
180  182          );
187  189
188  190          return Optional.ofNullable(
189  191              criteriaBuilder.equal(
190      -                root.join("application", JoinType.LEFT).get("codeIssuer"),
     192 +                root.join(APPLICATION, JoinType.LEFT).get("codeIssuer"),
191  193                  dataSourceCriteria.getApplicationCodeIssuer()
192  194              )
193  195          );
200  202
201  203          return Optional.ofNullable(
202  204              criteriaBuilder.equal(
203      -                root.join("application", JoinType.LEFT).get("trigram"),
     205 +                root.join(APPLICATION, JoinType.LEFT).get("trigram"),
204  206                  dataSourceCriteria.getApplicationTrigram()
205  207              )
206  208          );
213  215
214  216          return Optional.ofNullable(
215  217              criteriaBuilder.equal(
216      -                root.join("application", JoinType.LEFT).get("trigramIssuer"),
     218 +                root.join(APPLICATION, JoinType.LEFT).get("trigramIssuer"),
217  219                  dataSourceCriteria.getApplicationTrigramIssuer()
218  220              )
219  221          );
```

3

...cgen/bigdata/catalog/repositories/jpa/specification/JpaCdoDomainSpecificationBuilder.java

```
8    8   import org.springframework.data.jpa.domain.Specification;
9    9
10   10  public class JpaCdoDomainSpecificationBuilder {
     11 +    private JpaCdoDomainSpecificationBuilder(){
     12 +        /* This is a utility class and should not be instantiated */
     13 +    }
11   14
12   15      public static Specification<JpaCdoDomain> textSearch(String text) {
13   16          return (Root<JpaCdoDomain> root, CriteriaQuery<?> query, CriteriaBuilder
     builder) ->
```

3

.../socgen/bigdata/catalog/repositories/jpa/specification/JpaEntitySpecificationBuilder.java

```
12   12  import org.springframework.util.StringUtils;
13   13
14   14  public class JpaEntitySpecificationBuilder {
```

```
+     private JpaEntitySpecificationBuilder(){
+         /*  This is a utility class and should not be instantiated */
+     }

      public static BooleanExpression fullTextSearch(String text) {
          return ((text == null) || text.isEmpty())
```

3

...en/bigdata/catalog/repositories/jpa/specification/JpaEnvironmentSpecificationBuilder.java

```
  import org.springframework.data.jpa.domain.Specification;

  public class JpaEnvironmentSpecificationBuilder {
+     private JpaEnvironmentSpecificationBuilder(){
+         /*  This is a utility class and should not be instantiated */
+     }

      public static Specification<JpaEnvironment> textSearch(String text) {
          return (Root<JpaEnvironment> root, CriteriaQuery<?> query, CriteriaBuilder
  builder) ->
```

3

...gen/bigdata/catalog/repositories/jpa/specification/JpaExpositionSpecificationBuilder.java

```
  import org.springframework.data.jpa.domain.Specification;

  public class JpaExpositionSpecificationBuilder {
+     private JpaExpositionSpecificationBuilder(){
+         /* This is a utility class and should not be instantiated */
+     }

      public static Specification<JpaExposition> textSearch(String text) {
          return (Root<JpaExposition> root, CriteriaQuery<?> query, CriteriaBuilder
  builder) ->
```

3

...cgen/bigdata/catalog/repositories/jpa/specification/JpaLifecycleSpecificationBuilder.java

```
  import org.springframework.data.jpa.domain.Specification;

  public class JpaLifecycleSpecificationBuilder {
+     private JpaLifecycleSpecificationBuilder(){
+
+     }

      public static Specification<JpaLifecycle> textSearch(String text) {
          return (Root<JpaLifecycle> root, CriteriaQuery<?> query, CriteriaBuilder
  builder) ->
```

6

...in/java/com/socgen/bigdata/catalog/repositories/legalentity/LegalEntitySpecification.java

```
18  18
19  19      private LegalEntityCriteria legalEntityCriteria;
20  20
    21  +   private static final String EXTERNAL_COMPANY="externalCompany";
    22  +
21  23      public LegalEntitySpecification(LegalEntityCriteria legalEntityCriteria) {
22  24          this.legalEntityCriteria = legalEntityCriteria;
23  25      }
48  50              return Optional.empty();
49  51          }
50  52          return Optional.ofNullable(
51      -   builder.in(root.get("externalCompany")).value(legalEntityCriteria.getExternalCompanies())
    53  +   builder.in(root.get(EXTERNAL_COMPANY)).value(legalEntityCriteria.getExternalCompanies())
52  54          );
53  55      }
54  56
57  59              return Optional.empty();
58  60          }
59  61          return Optional.ofNullable(
60      -           builder.or(builder.isFalse(root.get("externalCompany")), builder.isNull(root.get("externalCompany")))
    62  +           builder.or(builder.isFalse(root.get(EXTERNAL_COMPANY)), builder.isNull(root.get(EXTERNAL_COMPANY)))
61  63          );
62  64      }
63  65
```

12

src/main/java/com/socgen/bigdata/catalog/resources/v2/AuditHistoryV2Resource.java

```
7   7   import io.swagger.v3.oas.annotations.Operation;
8   8   import io.swagger.v3.oas.annotations.Parameter;
9   9   import java.util.List;
10  -   import lombok.AllArgsConstructor;
11  10  import lombok.Data;
12  -   import lombok.NoArgsConstructor;
13  11  import lombok.extern.slf4j.Slf4j;
14  -   import org.springframework.beans.factory.annotation.Autowired;
15  12  import org.springframework.http.ResponseEntity;
16  13  import org.springframework.security.access.prepost.PreAuthorize;
17  14  import org.springframework.web.bind.annotation.*;
18  15
19  16  @RequestMapping("/v2/api/audit-history")
```

```
20  17   @RestController
21     - @NoArgsConstructor
22     - @AllArgsConstructor
23  18   @Data
24  19   @Slf4j
25  20   @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
26  21   public class AuditHistoryV2Resource {
27  22
28     - @Autowired
29     - private AuditHistoryService auditHistoryService;
    23 + private final AuditHistoryService auditHistoryService;
    24 +
    25 + public AuditHistoryV2Resource(AuditHistoryService auditHistoryService){
    26 +     this.auditHistoryService=auditHistoryService;
    27 + }
30  28
31  29       @Operation(description = "Get history entry")
32  30       @GetMapping(produces = APPLICATION_JSON_VALUE, value = "{id}")
```

8 src/main/java/com/socgen/bigdata/catalog/resources/v2/BuSuResource.java

```
17  17   import io.swagger.v3.oas.annotations.tags.Tag;
18  18   import java.util.List;
19  19   import lombok.extern.slf4j.Slf4j;
20     - import org.springframework.beans.factory.annotation.Autowired;
21  20   import org.springframework.security.access.prepost.PreAuthorize;
22  21   import org.springframework.web.bind.annotation.GetMapping;
23  22   import org.springframework.web.bind.annotation.RequestMapping;
38  37   @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
39  38   public class BuSuResource {
40  39
41     - @Autowired
42     - private OrganizationalUnitService organizationalUnitService;
    40 + private final OrganizationalUnitService organizationalUnitService;
    41 +
    42 + public BuSuResource(OrganizationalUnitService organizationalUnitService){
    43 +     this.organizationalUnitService=organizationalUnitService;
    44 + }
43  45
44  46       @GetMapping(produces = APPLICATION_JSON_VALUE)
45  47       @Operation(
```

8

src/main/java/com/socgen/bigdata/catalog/resources/v2/BusinessDataV2Resource.java

```
15  15   import io.swagger.v3.oas.annotations.responses.ApiResponses;
16  16   import java.util.List;
17  17   import lombok.extern.slf4j.Slf4j;
18     - import org.springframework.beans.factory.annotation.Autowired;
19  18   import org.springframework.http.ResponseEntity;
20  19   import org.springframework.security.access.prepost.PreAuthorize;
```

```
21  20  import org.springframework.web.bind.annotation.GetMapping;
31  30  @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
32  31  public class BusinessDataV2Resource {
33  32
34      -     @Autowired
35      -     private BusinessDataService businessDataService;
    33  +     private final BusinessDataService businessDataService;
    34  +
    35  +     public BusinessDataV2Resource(BusinessDataService businessDataService){
    36  +         this.businessDataService=businessDataService;
    37  +     }
36  38
37  39        @GetMapping(produces = APPLICATION_JSON_VALUE)
38  40        @Operation(
```

14

src/main/java/com/socgen/bigdata/catalog/resources/v2/CdoDomainV2Resource.java

```
2   2
3   3   import static org.springframework.http.MediaType.APPLICATION_JSON_VALUE;
4   4   import static org.springframework.http.MediaType.APPLICATION_PROBLEM_JSON_VALUE;
5       -
6   5   import com.socgen.apibank.openapi.customizer.ProblemResponseReference;
7   6   import com.socgen.bigdata.catalog.helpers.ExcelHelperConstant;
8   7   import com.socgen.bigdata.catalog.helpers.PaginationHelper;
23  22  import io.swagger.v3.oas.annotations.tags.Tag;
24  23  import java.util.List;
25  24  import lombok.extern.slf4j.Slf4j;
26      - import org.springframework.beans.factory.annotation.Autowired;
27  25  import org.springframework.beans.factory.parsing.Problem;
28  26  import org.springframework.data.domain.Pageable;
29  27  import org.springframework.data.web.PageableDefault;
58  56  @Validated
59  57  public class CdoDomainV2Resource {
60  58
61      -     @Autowired
62      -     private CdoDomainService cdoDomainService;
63  59
64      -     @Autowired
65      -     private UserService userService;
    60  +     private final CdoDomainService cdoDomainService;
    61  +     private final UserService userService;
    62  +
    63  +     public CdoDomainV2Resource(CdoDomainService cdoDomainService, UserService
      userService){
    64  +         this.cdoDomainService = cdoDomainService;
    65  +         this.userService = userService;
    66  +
    67  +     }
66  68
67  69        @Operation(description = "List all cdo domains")
```

```
68  70      @GetMapping(produces = APPLICATION_JSON_VALUE)
```

7 src/main/java/com/socgen/bigdata/catalog/resources/v2/CollibraResource.java

```
3   3   import com.socgen.bigdata.catalog.services.KearService;
4   4   import io.swagger.v3.oas.annotations.Operation;
5   5   import lombok.extern.slf4j.Slf4j;
6       - import org.springframework.beans.factory.annotation.Autowired;
7   6   import org.springframework.http.ResponseEntity;
8   7   import org.springframework.security.access.prepost.PreAuthorize;
9   8   import org.springframework.web.bind.annotation.RequestMapping;
19  18  @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v1')")
20  19  public class CollibraResource {
21  20
22      - @Autowired
23      - private KearService kearService;
    21  + private final KearService kearService;
    22  + public CollibraResource(KearService kearService){
    23  +     this.kearService=kearService;
    24  + }
24  25
25  26      @Operation(description = "load collibra business data in table
    business_dictionary from remote api and save them")
26  27      @RequestMapping(value = "/load-collibra-business-data", method = GET, produces
    = APPLICATION_JSON_VALUE)
```

12 src/main/java/com/socgen/bigdata/catalog/resources/v2/CountryResource.java

```
5   5   import io.swagger.v3.oas.annotations.Operation;
6   6   import io.swagger.v3.oas.annotations.Parameter;
7   7   import java.util.List;
8       - import lombok.AllArgsConstructor;
9   8   import lombok.Data;
10      - import lombok.NoArgsConstructor;
11  9   import lombok.extern.slf4j.Slf4j;
12      - import org.springframework.beans.factory.annotation.Autowired;
13  10  import org.springframework.security.access.prepost.PreAuthorize;
14  11  import org.springframework.web.bind.annotation.GetMapping;
15  12  import org.springframework.web.bind.annotation.RequestMapping;
18  15
19  16  @RequestMapping("/v2/api/countries")
20  17  @RestController
21      - @NoArgsConstructor
22      - @AllArgsConstructor
23  18  @Data
24  19  @Slf4j
25  20  @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
26  21  public class CountryResource {
27  22
28      - @Autowired
29      - private CountryService countryService;
```

```
+     private final CountryService countryService;
+
+     public CountryResource(CountryService countryService){
+         this.countryService=countryService;
+     }

      @Operation(description = "get list of countries by the criteria")
      @GetMapping(value = "/search")
```

8 src/main/java/com/socgen/bigdata/catalog/resources/v2/DxSearchV2Resource.java

```
  import com.socgen.bigdata.catalog.services.DxSearchService;
  import io.swagger.v3.oas.annotations.Operation;
- import org.springframework.beans.factory.annotation.Autowired;
  import org.springframework.http.ResponseEntity;
  import org.springframework.security.access.prepost.PreAuthorize;
  import org.springframework.web.bind.annotation.RequestMapping;
  @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
  public class DxSearchV2Resource {

-     @Autowired
-     private DxSearchService dxSearchService;
+     private final DxSearchService dxSearchService;
+
+     public DxSearchV2Resource(DxSearchService dxSearchService){
+         this.dxSearchService=dxSearchService;
+     }

      @PreAuthorize("hasAuthority('api.sgdata-catalog_technical-operation')")
      @Operation(description = "index datasets on dx-search service")
```

11

src/main/java/com/socgen/bigdata/catalog/resources/v2/EntityLegalResource.java

```
  import io.swagger.v3.oas.annotations.Operation;
  import io.swagger.v3.oas.annotations.Parameter;
  import java.util.List;
- import lombok.AllArgsConstructor;
  import lombok.Data;
- import lombok.NoArgsConstructor;
  import lombok.extern.slf4j.Slf4j;
- import org.springframework.beans.factory.annotation.Autowired;
  import org.springframework.security.access.prepost.PreAuthorize;
  import org.springframework.web.bind.annotation.GetMapping;
  import org.springframework.web.bind.annotation.RequestMapping;

  @RequestMapping("/v2/api/legalEntities")
  @RestController
- @NoArgsConstructor
- @AllArgsConstructor
```

```
@Data
@Slf4j
@PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
public class EntityLegalResource {

-    @Autowired
-    private LegalEntityService legalEntityService;
+    private final LegalEntityService legalEntityService;
+    public EntityLegalResource(LegalEntityService legalEntityService){
+        this.legalEntityService=legalEntityService;
+    }

     @Operation(description = "get list of legal entities by the criteria")
     @GetMapping(value = "/search")
```

8 src/main/java/com/socgen/bigdata/catalog/resources/v2/ExportResource.java

```
import java.util.Arrays;
import javax.servlet.http.HttpServletResponse;
import org.apache.commons.lang3.NotImplementedException;
- import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
@PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
public class ExportResource {

-    @Autowired
-    private DatasetExportService datasetExportService;
+    private final DatasetExportService datasetExportService;
+
+    public ExportResource(DatasetExportService datasetExportService){
+        this.datasetExportService=datasetExportService;
+    }

     @GetMapping(value = "/datasets")
     @Operation(
```

8 src/main/java/com/socgen/bigdata/catalog/resources/v2/FilterV2Resource.java

```
import io.swagger.v3.oas.annotations.Parameter;
import java.util.List;
import lombok.extern.slf4j.Slf4j;
- import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.data.web.SortDefault;
@PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
public class FilterV2Resource {

-    @Autowired
```

```
33      -     private FilterService filterService;
        31 +  private final FilterService filterService;
        32 +
        33 +  public FilterV2Resource(FilterService filterService){
        34 +      this.filterService=filterService;
        35 +  }
34      36
35      37    @Operation(description = "List all Refresh frequencies")
36      38    @GetMapping(produces = APPLICATION_JSON_VALUE, value = "refresh-frequencies")
```

12

src/main/java/com/socgen/bigdata/catalog/resources/v2/RestrictionResource.java

```
4       4   import com.socgen.bigdata.catalog.services.RestrictionService;
5       5   import io.swagger.v3.oas.annotations.Operation;
6       6   import io.swagger.v3.oas.annotations.Parameter;
7       -   import lombok.AllArgsConstructor;
8       7   import lombok.Data;
9       -   import lombok.NoArgsConstructor;
10      8   import lombok.extern.slf4j.Slf4j;
11      -   import org.springframework.beans.factory.annotation.Autowired;
12      9   import org.springframework.security.access.prepost.PreAuthorize;
13      10  import org.springframework.web.bind.annotation.*;
14      11
15      12  @RequestMapping("/v2/api/restrictions")
16      13  @RestController
17      -   @NoArgsConstructor
18      -   @AllArgsConstructor
19      14  @Data
20      15  @Slf4j
21      16  @PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
22      17  public class RestrictionResource {
23      18
24      -       @Autowired
25      -       private RestrictionService restrictionService;
        19 +    private final RestrictionService restrictionService;
        20 +
        21 +    public RestrictionResource(RestrictionService restrictionService){
        22 +        this.restrictionService=restrictionService;
        23 +    }
26      24
27      25      @Operation(description = "Get all restrictions for a given dataset")
28      26      @GetMapping(value = "/{id}")
```

9 src/main/java/com/socgen/bigdata/catalog/resources/v2/TagV2Resource.java

```
11      11  import io.swagger.v3.oas.annotations.Parameter;
12      12  import java.util.List;
13      13  import lombok.extern.slf4j.Slf4j;
14      -   import org.springframework.beans.factory.annotation.Autowired;
15      14  import org.springframework.data.domain.Page;
```

```
import org.springframework.data.domain.PageRequest;
import org.springframework.http.ResponseEntity;
@PreAuthorize("hasAuthority('SCOPE_api.sgdata-catalog.v2')")
public class TagV2Resource {

-    @Autowired
-    private TagService tagService;
+
+    private final TagService tagService;
+
+    public TagV2Resource(TagService tagService){
+        this.tagService=tagService;
+    }

    @Operation(description = "List all tags from all categories")
    @GetMapping(produces = APPLICATION_JSON_VALUE)
```

19 src/main/java/com/socgen/bigdata/catalog/security/OAuth2BaseProperties.java

```
@@ -1,43 +1,28 @@
package com.socgen.bigdata.catalog.security;

+ import lombok.Getter;
import java.util.ArrayList;
import java.util.List;
-
+ @Getter
public class OAuth2BaseProperties {

    protected String accessTokenUri;
    protected String clientId;
    protected String clientSecret;
    protected List<String> scopes = new ArrayList<>();

-    public String getAccessTokenUri() {
-        return accessTokenUri;
-    }
-
    public void setAccessTokenUri(String accessTokenUri) {
        this.accessTokenUri = accessTokenUri;
    }

-    public String getClientId() {
-        return clientId;
-    }
-
    public void setClientId(String clientId) {
        this.clientId = clientId;
    }

-    public String getClientSecret() {
```

```
30        -        return clientSecret;
31        -    }
32        -
33   22       public void setClientSecret(String clientSecret) {
34   23           this.clientSecret = clientSecret;
35   24       }
36   25
37        -    public List<String> getScopes() {
38        -        return scopes;
39        -    }
40        -
41   26       public void setScopes(List<String> scopes) {
42   27           this.scopes = scopes;
43   28       }
```

2

src/main/java/com/socgen/bigdata/catalog/security/UserAuthoritiesServiceImpl.java

```
10   10   @Service
11   11   public class UserAuthoritiesServiceImpl {
12   12
13        -    private JpaPersonRepository personRepository;
     13   +    private final JpaPersonRepository personRepository;
14   14
15   15       public UserAuthoritiesServiceImpl(JpaPersonRepository personRepository) {
16   16           this.personRepository = personRepository;
```

6 src/main/java/com/socgen/bigdata/catalog/services/AuditHistoryService.java

```
49   49           String objectType,
50   50           String userId,
51   51           Boolean isAutomaticallyGenerated,
52        -       List ignoreProperties
     52   +       List<String> ignoreProperties
53   53       ) {
54        -       Map oldValueMap = JsonDtoCompare.flatten(oldValue);
55        -       Map newValueMap = JsonDtoCompare.flatten(newValue);
     54   +       Map<String, Object> oldValueMap = JsonDtoCompare.flatten(oldValue);
     55   +       Map<String, Object> newValueMap = JsonDtoCompare.flatten(newValue);
56   56           JpaAuditHistroy jpaAuditHistroySaved = null;
57   57           try {
58   58               String deltaDto = JsonDtoCompare.diffMap(oldValueMap, newValueMap,
     ignoreProperties);
```

4 src/main/java/com/socgen/bigdata/catalog/services/BusinessDataService.java

```
127  127      }
128  128
129  129      public List<FilterItemDto> searchDxBusinessData(String text, List<String> ids)
     {
```

```
130 -         Boolean textNullOrEmpty = text == null || text.isEmpty();
131 -         Boolean idsNullOrEmpty = ids == null || ids.isEmpty();
    130 +         boolean textNullOrEmpty = (text == null || text.isEmpty());
    131 +         boolean idsNullOrEmpty = (ids == null || ids.isEmpty()) ;
132 132           List<Long> idLists = new ArrayList<>();
133 133           if (!idsNullOrEmpty) {
134 134               idLists = ids.stream().map(Long::parseLong).toList();
```

2 src/main/java/com/socgen/bigdata/catalog/services/DxSearchService.java

```
67  67                      );
68  68
69  69                      datasetsPartition.forEach(datasetList -> {
70  -                           Boolean updateResult = updateBulkDatasetAssetsDxSearch(datasetList, true);
    70  +                       boolean updateResult = updateBulkDatasetAssetsDxSearch(datasetList, true);
71  71                          if (!updateResult) {
72  72                              throw new RuntimeException("Update failed");
73  73                          }
```

15 src/main/java/com/socgen/bigdata/catalog/services/EntityService.java

```
88  88  import com.socgen.bigdata.catalog.models.jpa.JpaProcessReferential;
89  89  import com.socgen.bigdata.catalog.models.jpa.JpaRelatedApi;
90  90  import com.socgen.bigdata.catalog.models.jpa.JpaRelatedConsoleProject;
91  -   import com.socgen.bigdata.catalog.models.jpa.LegalEntity;
92  91  import com.socgen.bigdata.catalog.models.jpa.OrganizationalUnit;
93  92  import com.socgen.bigdata.catalog.models.jpa.QJpaEntity;
94  93  import com.socgen.bigdata.catalog.models.jpa.Tag;
149 148 import java.util.Collection;
150 149 import java.util.Collections;
151 150 import java.util.Comparator;
152 -   import java.util.Date;
153 151 import java.util.HashMap;
154 152 import java.util.HashSet;
155 153 import java.util.LinkedHashMap;
201 199     public static final String ONE_OR_MORE_VALIDATION_ERRORS_OCCURRED = "One or more validation errors occurred.";
202 200
203 201     private static final String ADD_UPDATE_ERROR_KEY_BU_SU_OWNERSHIP = "buSuOwnership";
    202 +   private static final String DATASET="dataset";
204 203
205 204     @Autowired
206 205     private CdoDomainService domainService;
399 398         addUserViewProof(entity.getId());
400 399         EntityDetailDto entityDetailDto = entityMapper.toDetailDto(entity);
401 400         entityDetailDto.setRelatedFeedbacks(
402 -               feedbackService.getFeedbackByObject(entity.getFunctionalId() + "", "dataset")
```

```
+            feedbackService.getFeedbackByObject(entity.getFunctionalId() + "", DATASET)
        );
        setRightsToDto(entity.getId(), entityDetailDto);
        return entityDetailDto;
            oldMap,
            newMap,
            jpaEntity.getFunctionalId(),
-           "dataset",
+           DATASET,
            userService.getMail(),
            false,
            Arrays.asList("lastUpdateDate", "updatedBy", "editable", "removable")
        }

        newEntityDetailDto.setRelatedFeedbacks(
-           feedbackService.getFeedbackByObject(jpaEntity.getFunctionalId(), "dataset")
+           feedbackService.getFeedbackByObject(jpaEntity.getFunctionalId(), DATASET)
        );

        return newEntityDetailDto;
                        oldEntityDetailDto,
                        newEntityDetailDto,
                        dataset.getFunctionalId(),
-                       "dataset",
+                       DATASET,
                        null,
                        true,
                        null
                oldDataSourceAuditDto,
                newDataSourceAuditDto,
                updatedDataset.getFunctionalId(),
-               "dataset",
+               DATASET,
                userService.getMail(),
                false,
                null
                oldEntityDetailDto,
                newEntityDetailDto,
                datasetSaved.getFunctionalId(),
-               "dataset",
+               DATASET,
                null,
                true,
                null
```

28 src/main/java/com/socgen/bigdata/catalog/services/KearService.java

```
import java.util.Set;
import java.util.stream.Collectors;
```

```
18  18   import lombok.extern.slf4j.Slf4j;
19      - import org.springframework.beans.factory.annotation.Autowired;
20  19   import org.springframework.beans.factory.annotation.Value;
21  20   import org.springframework.core.ParameterizedTypeReference;
22  21   import org.springframework.http.HttpEntity;
32  31   @Service
33  32   @Slf4j
34  33   public class KearService {
    34  +     private static final String VALUE_FIELD="value";
35  35
36      -     @Autowired
37      -     private JpaKearRepository jpaKearRepository;
    36  +     private final JpaKearRepository jpaKearRepository;
38  37
39      -     @Autowired
40      -     private JpaBusinessDataRepository jpaBusinessDataRepository;
    38  +     private final JpaBusinessDataRepository jpaBusinessDataRepository;
41  39
42  40        @Value("#{'${service.business-concepts-collibra.url}'.trim()}")
43  41        private String businessConceptsCollibraUrl;
44  42
45      -     @Autowired
46      -     private OAuth2RestTemplate collibraBusinessConceptsRestTemplate;
    43  +
    44  +     private final OAuth2RestTemplate collibraBusinessConceptsRestTemplate;
    45  +
    46  +     public KearService(
    47  +         JpaKearRepository jpaKearRepository,
    48  +         JpaBusinessDataRepository jpaBusinessDataRepository,
    49  +         OAuth2RestTemplate collibraBusinessConceptsRestTemplate
    50  +     ) {
    51  +         this.jpaKearRepository = jpaKearRepository;
    52  +         this.jpaBusinessDataRepository = jpaBusinessDataRepository;
    53  +         this.collibraBusinessConceptsRestTemplate =
      collibraBusinessConceptsRestTemplate;
    54  +     }
47  55
48  56
49  57        private HttpEntity getCollibraHeaderEntity() {
118 126       public static BusinessDataDto mapCollibraToBusinessDataDto(JsonNode
      businessDataJson) {
119 127           BusinessDataDto dto = new BusinessDataDto();
120 128           dto.setCollibraId(businessDataJson.path("assetId").asText(null));
121     -
      dto.setBusinessDataGlobalId(businessDataJson.path("kearId").path(0).path("value").a
      sText(null));
    129 +
      dto.setBusinessDataGlobalId(businessDataJson.path("kearId").path(0).path(VALUE_FIEL
      D).asText(null));
122 130           dto.setName(businessDataJson.path("assetNameEn").asText(null));
123     -
      dto.setDefinition(businessDataJson.path("definitionEn").path(0).path("value").asTex
```

```
t(null));
+ dto.setDefinition(businessDataJson.path("definitionEn").path(0).path(VALUE_FIELD).asText(null));
  dto.setValidationStatus(businessDataJson.path("status").path(0).path("name").asText(null));
- dto.setComment(businessDataJson.path("informationEn").path(0).path("value").asText(null));
+ dto.setComment(businessDataJson.path("informationEn").path(0).path(VALUE_FIELD).asText(null));
        dto.setOrigin("COLLIBRA");
        dto.setLastUpdateDate(new Date());
```

1 src/main/java/com/socgen/bigdata/catalog/services/board/BoardUtils.java

```
  import com.socgen.bigdata.catalog.models.jpa.JpaCountry;
  import com.socgen.bigdata.catalog.models.jpa.JpaDatasetRelationship;
  import com.socgen.bigdata.catalog.models.jpa.JpaEntity;
- import com.socgen.bigdata.catalog.models.jpa.LegalEntity;
  import com.socgen.bigdata.catalog.models.jpa.Tag;
  import com.socgen.bigdata.catalog.models.jpa.person.JpaPerson;
  import java.math.BigDecimal;
```

2

...main/java/com/socgen/bigdata/catalog/services/board/dataset/DatasetBoardAgGridConfig.java

```
  import com.socgen.bigdata.catalog.enums.Exposition;
  import com.socgen.bigdata.catalog.enums.board.BoardAgGridType;
  import com.socgen.bigdata.catalog.enums.board.dataset.DatasetBoardAgGridSection;
- import com.socgen.bigdata.catalog.models.jpa.JpaBusinessData;
  import com.socgen.bigdata.catalog.models.jpa.JpaEntity;
  import com.socgen.bigdata.catalog.services.board.BoardComputation;
  import com.socgen.bigdata.catalog.services.board.BoardUtils;
  import java.util.ArrayList;
  import java.util.List;
  import java.util.Objects;
- import java.util.Optional;
  import java.util.function.Function;
  import javax.annotation.PostConstruct;
  import lombok.Getter;
```

8

src/main/java/com/socgen/bigdata/catalog/services/mappers/CdoDomainMapperCache.java

```
  import com.socgen.bigdata.catalog.models.dto.CdoDomainDto;
  import com.socgen.bigdata.catalog.models.jpa.JpaCdoDomain;
```

```
5   import java.util.List;
6 - import org.springframework.beans.factory.annotation.Autowired;
7   import org.springframework.stereotype.Service;
8
9   @Service
10  public class CdoDomainMapperCache {
11
12 -     @Autowired
13 -     private CdoDomainMapper cdoDomainMapper;
    +     private final CdoDomainMapper cdoDomainMapper;
    +
    +     public CdoDomainMapperCache(CdoDomainMapper cdoDomainMapper){
    +         this.cdoDomainMapper=cdoDomainMapper;
    +     }
14
15      public CdoDomainDto toDto(JpaCdoDomain cdoDomain, List<Object[]> allEntityCountByCdoDomain) {
16          return cdoDomainMapper.toDto(cdoDomain, allEntityCountByCdoDomain);
```

2 src/main/java/com/socgen/bigdata/catalog/services/mappers/EntityMapper.java

```
521
522         if (!CollectionUtils.isEmpty(dataSources)) {
523             countDataSourceData += dataSources.size();
524 -
525 -           final Set<String> headers = new HashSet<>();
526             final String ON_PREM_LUCID = "ON_PREM_LUCID";
527             final String AZURE_DATA_LAKE = "AZURE_DATA_LAKE";
528
```

16

src/main/java/com/socgen/bigdata/catalog/services/snowflake/SnowflakeImportService.java

```
34      private final ApplicationService applicationService;
35      private final ScalityService scalityService;
36      private final SnowflakeImportAuditRepository auditRepository;
    +   private static final String CATALOG_NAME="CATALOG_NAME";
    +   private static final String VALUE_REQUIRED_MESSAGE = "Value required";
37
38      public SnowflakeImportService(
39          SnowflakeDataImportRepository importRepository,
56              return ImportResult.aborted(
57                  "Empty file",
58                  List.of(
59 -                    new ImportErrorItem(parsed.firstDataLineNumber, "CATALOG_NAME", "", "Value required"),
60 -                    new ImportErrorItem(parsed.firstDataLineNumber, "SCHEMA_NAME", "", "Value required")
    +                   new ImportErrorItem(parsed.firstDataLineNumber, CATALOG_NAME, "", VALUE_REQUIRED_MESSAGE),
```

```
+                    new ImportErrorItem(parsed.firstDataLineNumber, "SCHEMA_NAME", "", VALUE_REQUIRED_MESSAGE)
                    ),
                    s3Key
                );

                if (catalog.isEmpty() || schema.isEmpty()) {
                    if (catalog.isEmpty()) {
-                    errors.add(new ImportErrorItem(currentLine, "CATALOG_NAME", "", "Value required"));
+                    errors.add(new ImportErrorItem(currentLine, CATALOG_NAME, "", VALUE_REQUIRED_MESSAGE));
                    }
                    if (schema.isEmpty()) {
-                    errors.add(new ImportErrorItem(currentLine, "SCHEMA_NAME", "", "Value required"));
+                    errors.add(new ImportErrorItem(currentLine, CATALOG_NAME, "", VALUE_REQUIRED_MESSAGE));
                    }
                    continue;
                }

                final RowKey key = new RowKey(catalog, schema);
                if (seenInFile.contains(key)) {
                    errors.add(
-                    new ImportErrorItem(currentLine, "CATALOG_NAME", catalog, "Duplicate catalog/schema in file")
+                    new ImportErrorItem(currentLine, CATALOG_NAME, catalog, "Duplicate catalog/schema in file")
                    );
                    continue;
                }
                    errors.add(
                        new ImportErrorItem(
                            currentLine,
-                           "CATALOG_NAME",
+                           CATALOG_NAME,
                            catalog,
                            "Invalid pattern – must match ENV_TRI_IRT"
                        )
                    errors.add(
                        new ImportErrorItem(
                            currentLine,
-                           "CATALOG_NAME",
+                           CATALOG_NAME,
                            catalog,
                            "Application not found for trigram=" + pr.tri + ", IRT=" + pr.irt
                        )
```